

14. ROS2 DDS

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. DDS Overview

DDS stands for Data Distribution Service. It was released and maintained by the Object Management Group (OMG) in 2004. It is a standard for data distribution/subscription specifically designed for real-time systems. It was first used by the U.S. Navy to solve compatibility issues with numerous software upgrades in complex ship network environments. It has now become a mandatory standard.

DDS emphasizes data-centricity and can provide rich Quality of Service (QoS) policies to ensure real-time, efficient, and flexible data distribution. It can meet various distributed real-time communication application requirements.

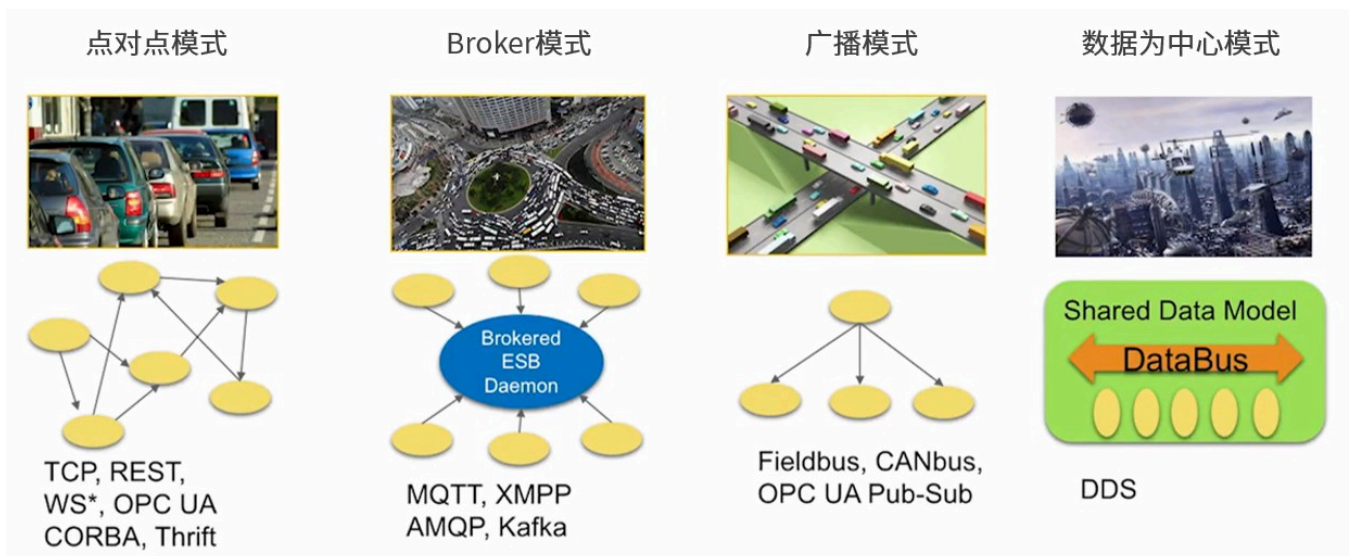
References:

- Fast DDS official documentation: [3.3.0](#)
- ROS2 official documentation DDS advanced features: [Click to jump](#)

2. Communication Models

The topic, service, and action communication we studied in previous courses all have their underlying communication implementation completed by DDS. It is like the neural network in the ROS robot system.

The core of DDS is communication. There are many communication models and software frameworks that can achieve communication. Here we list four commonly used models.



- The first one, **Point-to-Point Model**: Many clients connect to one server. Each communication requires establishing a connection between both parties. When communication nodes increase, the number of connections also increases. Moreover, each client needs to know the server's specific address and the services it provides. If the server address changes, all clients are affected.
- The second one, **Broker Model**: Optimized from the point-to-point model. The Broker centrally processes everyone's requests and further finds the role that can actually respond to the service. This

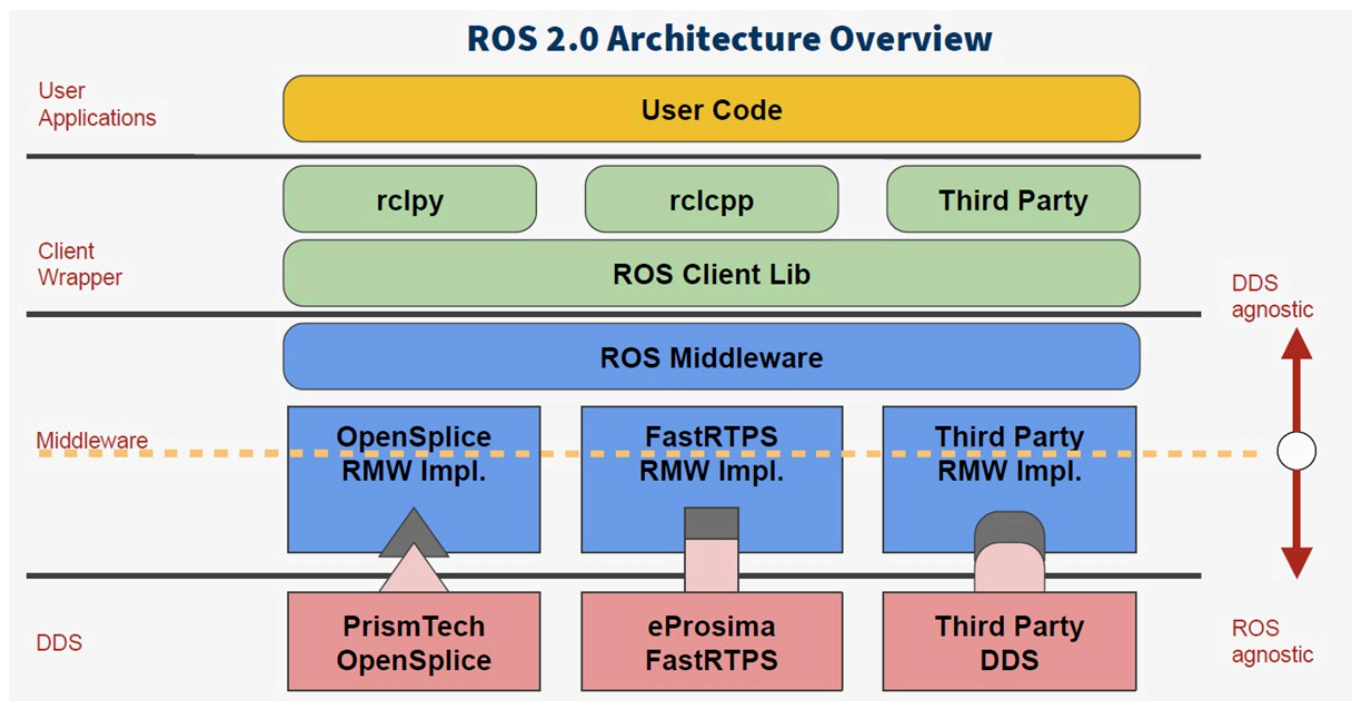
way, clients don't need to care about the server's specific address. However, the problem is obvious: the Broker is the core, and its processing speed affects the efficiency of all nodes. When the system scale grows to a certain level, the Broker becomes a performance bottleneck for the entire system. More troublesome is that if the Broker encounters an exception, it may cause the entire system to malfunction. The previous ROS1 system used an architecture similar to this.

- The third one, **Broadcast Model**: All nodes can broadcast messages on the channel, and all nodes can receive messages. This model solves the server address problem, and communication parties don't need to establish a separate connection. However, there are too many messages on the broadcast channel, and all nodes must care about every message, though many are actually irrelevant to them.
- The fourth one is the **Data-Centric DDS Model**. This model is similar to the broadcast model where all nodes can publish and subscribe to messages on the DataBus. However, its advancement lies in that the communication contains many parallel paths. Each node can only care about messages it is interested in and ignore messages it is not interested in. It's like a rotating hotpot where all kinds of delicious food are delivered on the DataBus. We only need to take what we want to eat. The rest is not our concern.

As you can see, among these communication models, DDS's advantages are more obvious.

3. DDS in ROS2

DDS's position in the ROS2 system is crucial. All upper-level construction is built on DDS. In this ROS2 architecture diagram, the blue and red parts are DDS.



Among the four main components of ROS, due to the addition of DDS, the comprehensive capability of the distributed communication system has been greatly improved. This way, during robot development, we don't need to worry about communication issues and can spend more time on application development in other areas.

4. Quality of Service (QoS) Policies

The basic structure in DDS is Domain, which binds various applications together for communication. Recall when we configured RDK and computer communication before, we configured the DOMAIN ID. That's the grouping definition for the global data space. Only nodes in the same DOMAIN group can communicate with each other. This avoids resource consumption by useless data.

Another important feature in DDS is Quality of Service (QoS).

QoS is a network transmission strategy. Applications specify the required network transmission quality behavior, and the QoS service tries to meet the customer's communication quality requirements as much as possible. It can be understood as a contract between data providers and receivers.

Policies are as follows:

- **DEADLINE** policy: Indicates that communication data must complete one communication within each deadline;
- **HISTORY** policy: Indicates a cache size for historical data;
- **RELIABILITY** policy: Indicates the data communication mode. Configuring as BEST_EFFORT means best-effort transmission mode. When network conditions are not good, data flow must be ensured, which may result in data loss. Configuring as RELIABLE means reliable mode, which tries to ensure data integrity during communication. We can choose the appropriate communication mode based on the application scenario;
- **DURABILITY** policy: Can be configured to ensure that late-joining nodes also receive some historical data. This allows new nodes to quickly adapt to the system.

5. Test Examples

5.1 Example 1 - Configure DDS via Command Line

- Open the first terminal and use the following command to publish a topic:

```
ros2 topic pub /chatter std_msgs/msg/Int32 "data: 66" --qos-reliability best_effort
```

```
yahboom@yahboom-virtual-machine:~$ ros2 topic pub /chatter std_msgs/msg/Int32 "data: 66" --qos-reliability best_effort
publisher: beginning loop
publishing #1: std_msgs.msg.Int32(data=66)
publishing #2: std_msgs.msg.Int32(data=66)
publishing #3: std_msgs.msg.Int32(data=66)
```

- Open another terminal and use a different QoS to print the topic. If the QoS policy used differs from that of the publisher, a warning will appear and topic data cannot be received normally:

```
ros2 topic echo /chatter --qos-reliability reliable
```

```
yahboom@yahboom-virtual-machine:~$ ros2 topic echo /chatter --qos-reliability reliable
[WARN] [1756266808.953062451] [_ros2cli_2810]: New publisher discovered on topic '/chatter', offering incompatible QoS. No messages will be received from it. Last incompatible policy: RELIABILITY
```

- You can receive topic data by using the same QoS policy as the topic publisher:

```
ros2 topic echo /chatter --qos-reliability best_effort
```

```
yahboom@yahboom-virtual-machine:~$ ros2 topic echo /chatter --qos-reliability best_effort
data: 66
---
data: 66
```

5.2 Example 2 - Write Topic Node with QoS Configuration

- Use the following command to create a new package:

```
ros2 pkg create learning_dds --build-type ament_python --dependencies rclpy std_msgs
```

- Create a new dds_controller_pub.py file as the publisher for topic communication. Fill in the following content:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
# Import QoS-related classes
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy

class ControllerPublisher(Node):
    def __init__(self, name):
        super().__init__(name)
        # 1. Configure QoS: Reliable transmission, keep last 1 piece of historical data
        self.qos_profile = QoSProfile(
            reliability=QoSReliabilityPolicy.RELIABLE, # Reliable transmission (retransmit
lost data)
            history=QoSHistoryPolicy.KEEP_LAST,        # Keep last N pieces of data
            depth=1                                     # Keep 1 piece of historical data
        )
        # 2. Create publisher: topic name/robot_cmd, message type String, QoS policy
        self.publisher = self.create_publisher(
            String,
            "/robot_cmd",
            self.qos_profile
        )
        # 3. Create timer: Send command once per second
        self.timer = self.create_timer(1.0, self.timer_callback)
        self.cmd_list = ["forward", "backward", "stop"] # Command list
        self.cmd_index = 0 # Command index, switch in cycle

    def timer_callback(self):
        # Cycle through commands (forward → backward → stop → forward...)
        current_cmd = self.cmd_list[self.cmd_index % 3]
        # Create message and fill data
        msg = String()
        msg.data = current_cmd
        # Publish message
```

```

        self.publisher.publish(msg)
        # Print log (show published command)
        self.get_logger().info(f"Publishing control command: {msg.data}")
        # Update command index
        self.cmd_index += 1

def main(args=None):
    # Initialize ROS2
    rclpy.init(args=args)
    # Create publisher node
    node = ControllerPublisher("robot_controller_pub")
    # Loop running node
    rclpy.spin(node)
    # Destroy node and shutdown ROS2
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()

```

- Subscriber code implementation. Create a new dds_robot_sub.py file and fill in the following subscriber implementation code:

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy

class RobotSubscriber(Node):
    def __init__(self, name):
        super().__init__(name)
        # 1. Configure QoS compatible with publisher
        self.qos_profile = QoSProfile(
            reliability=QoSReliabilityPolicy.BEST_EFFORT,
            history=QoSHistoryPolicy.KEEP_LAST,
            depth=1
        )
        # 2. Create subscriber: topic name/robot_cmd, callback function, QoS policy
        self.subscription = self.create_subscription(
            String,
            "/robot_cmd",
            self.cmd_callback, # Callback function to execute when data is received
            self.qos_profile
        )

    def cmd_callback(self, msg):
        # Callback function: Process received command
        self.get_logger().info(f"Received control command: {msg.data} → Execute corresponding action")

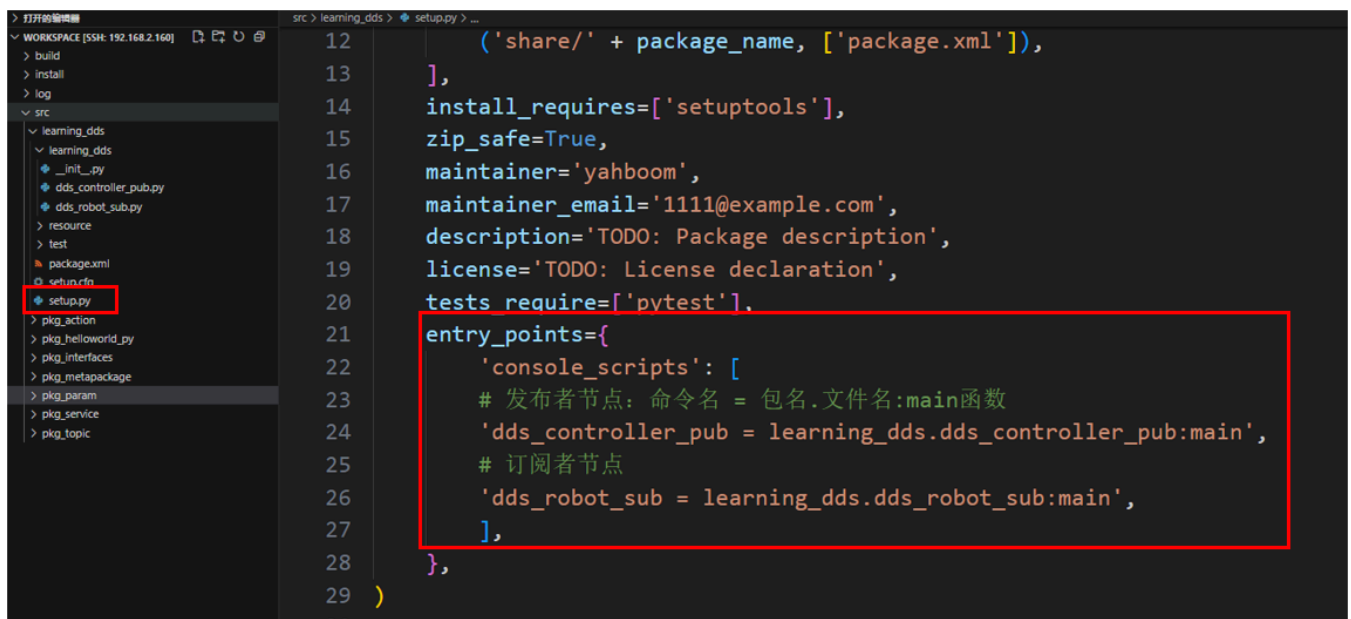
```

```
def main(args=None):
    rclpy.init(args=args)
    node = RobotSubscriber("robot_subscriber")
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

- Configure build file (setup.py):

```
entry_points={
    'console_scripts': [
        # Publisher node: command name = package name.filename:main function
        'dds_controller_pub = learning_dds.dds_controller_pub:main',
        # Subscriber node
        'dds_robot_sub = learning_dds.dds_robot_sub:main',
    ],
},
```



```
src > learning_dds > setup.py > ...
12     ('share/' + package_name, ['package.xml']),
13 ],
14 install_requires=['setuptools'],
15 zip_safe=True,
16 maintainer='yahboom',
17 maintainer_email='1111@example.com',
18 description='TODO: Package description',
19 license='TODO: License declaration',
20 tests_require=['pytest'],
21 entry_points={
22     'console_scripts': [
23         # 发布者节点: 命令名 = 包名.文件名:main函数
24         'dds_controller_pub = learning_dds.dds_controller_pub:main',
25         # 订阅者节点
26         'dds_robot_sub = learning_dds.dds_robot_sub:main',
27     ],
28 },
29 )
```

- Open a terminal in the workspace directory and build the package:

```
colcon build --packages-select learning_dds
```

```
colcon build` successful
/home/yahboom/workspace

yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 102x24
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select learning_dds
Starting >>> learning_dds
Finished <<< learning_dds [0.70s]

Summary: 1 package finished [0.90s]
yahboom@yahboom-virtual-machine:~/workspace$
```

- Set environment variables. Environment variables need to be set after each rebuild:

```
source install/setup.bash
```

- Run publisher and subscriber nodes:

```
ros2 run learning_dds dds_controller_pub
```

```
ros2 run learning_dds dds_robot_sub
```

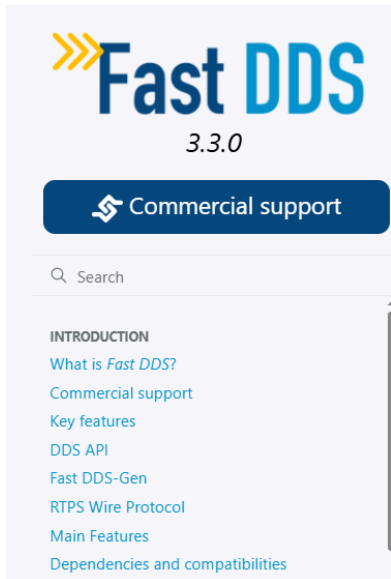
```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 109x15
[INFO] [1757057829.779997618] [robot_controller_pub]: 发布控制指令 : forward
[INFO] [1757057830.779293315] [robot_controller_pub]: 发布控制指令 : backward
[INFO] [1757057831.779651966] [robot_controller_pub]: 发布控制指令 : stop
[INFO] [1757057832.779404894] [robot_controller_pub]: 发布控制指令 : forward
[INFO] [1757057833.780227835] [robot_controller_pub]: 发布控制指令 : backward
[INFO] [1757057834.779559684] [robot_controller_pub]: 发布控制指令 : stop
[INFO] [1757057835.779722943] [robot_controller_pub]: 发布控制指令 : forward
[INFO] [1757057836.779981460] [robot_controller_pub]: 发布控制指令 : backward
[INFO] [1757057837.780282522] [robot_controller_pub]: 发布控制指令 : stop
[INFO] [1757057838.779935571] [robot_controller_pub]: 发布控制指令 : forward
[INFO] [1757057839.780166616] [robot_controller_pub]: 发布控制指令 : backward
[INFO] [1757057840.779678431] [robot_controller_pub]: 发布控制指令 : stop
[INFO] [1757057841.780123335] [robot_controller_pub]: 发布控制指令 : forward
[INFO] [1757057842.780290288] [robot_controller_pub]: 发布控制指令 : backward

yahboom@yahboom-virtual-machine: ~/workspace 109x15
yahboom@yahboom-virtual-machine:~/workspace$ source install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$ ros2 run learning_dds dds_robot_sub
[INFO] [1757057838.788915941] [robot_subscriber]: 接收控制指令 : forward → 执行对应动作
[INFO] [1757057839.780669612] [robot_subscriber]: 接收控制指令 : backward → 执行对应动作
[INFO] [1757057840.780217357] [robot_subscriber]: 接收控制指令 : stop → 执行对应动作
[INFO] [1757057841.780476044] [robot_subscriber]: 接收控制指令 : forward → 执行对应动作
[INFO] [1757057842.780689429] [robot_subscriber]: 接收控制指令 : backward → 执行对应动作
```

Robot Development Advanced:

- Generally, topic communication publisher and subscriber need to maintain the same QoS communication policy to avoid some hidden communication layer issues;
- QoS communication policy settings should be chosen based on actual application scenarios;
- If the application scenario involves special requirements at the communication layer such as drone

communication, encrypted communication, or real-time communication, find the Fast-DDS official manual in the tutorial introduction for more detailed DDS functionality explanations.



What is *Fast DDS*?



eProsima Fast DDS is a C++ implementation of the [DDS \(Data Distribution Service\) Specification](#), a protocol defined by the [Object Management Group \(OMG\)](#). The *eProsima Fast DDS* library provides both an Application Programming Interface (API) and a communication protocol that deploy a Data-Centric Publisher-Subscriber (DCPS) model, with the purpose of establishing efficient and reliable information distribution among Real-Time Systems.

Commercial support

Looking for commercial support? Write us to info@eprosima.com

Find more about us at [eProsima's webpage](#).

Key features